

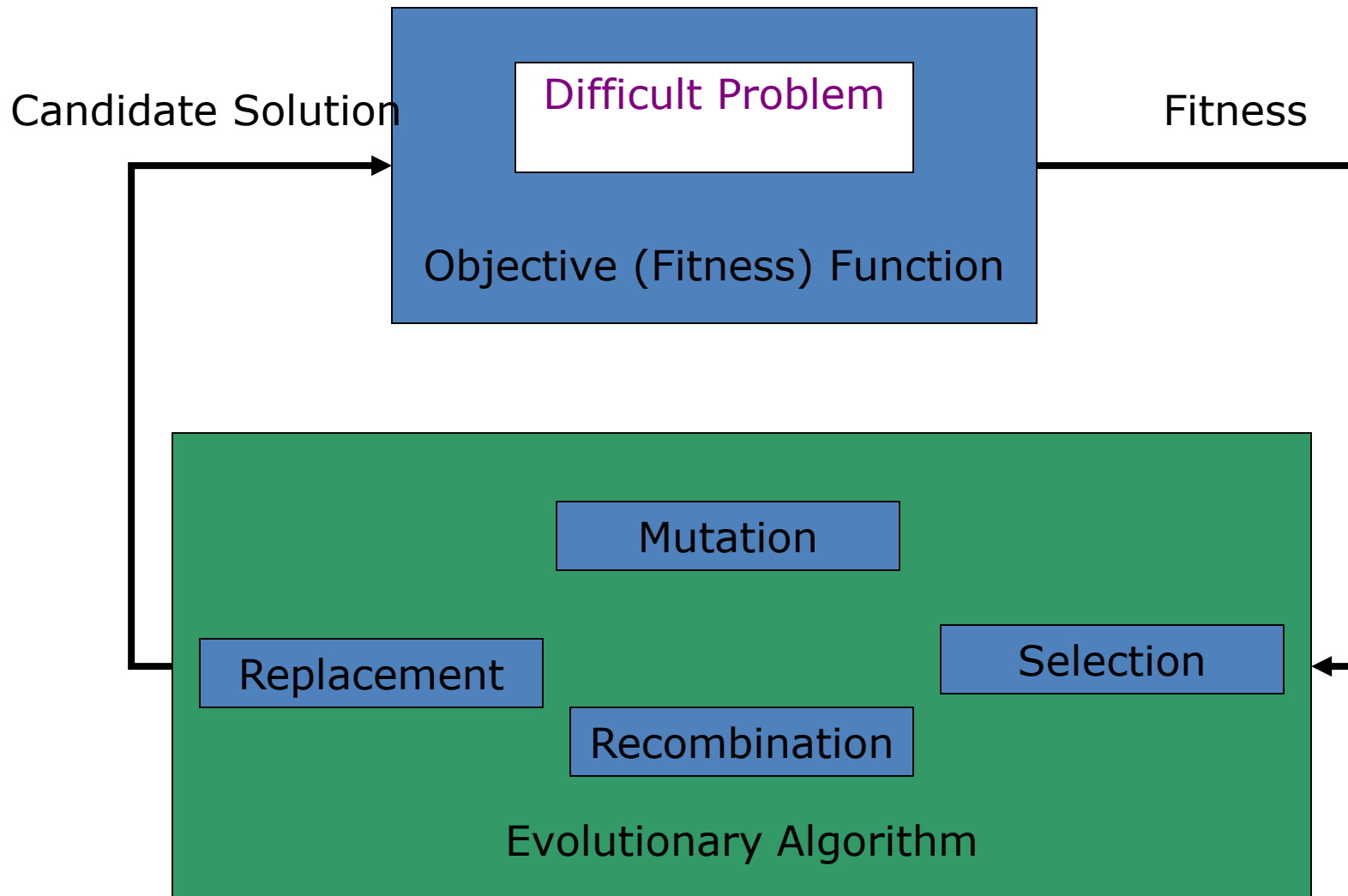
ECM3412/ECMM409
Nature Inspired Computation
Lecture 3

Basic Principles/Terms used in Evolutionary
Algorithms, Local and Population-based search and
Search Landscapes

Contents

- Basic Principles & Terms Used in EAs
- Simpler Algorithms
 - Hillclimbing
 - Local Search
- Search Landscapes
- Advantages of population-based search

Basic Principle of Evolutionary Algorithms



Some Terms Used in Evolutionary Computation

- **Algorithm Type**
 - Generational
 - Steady State
 - Elitist
- **Recombination (Crossover)**
 - Single-point
 - Multi-point
 - Uniform
- **Selection**
 - Rank-Based
 - Roulette Wheel
 - Tournament
- **Replacer**
 - Weakest
 - First weaker
- **Mutation**
 - Single Gene
 - Multiple Gene
- **Parameters**
 - Population size
 - Generations/iterations
 - Mutation rate
 - Crossover rate
 - Tournament size....
 - Etc

The Travelling Salesperson Problem

An example (hard) problem, for illustration

The Travelling Salesperson Problem

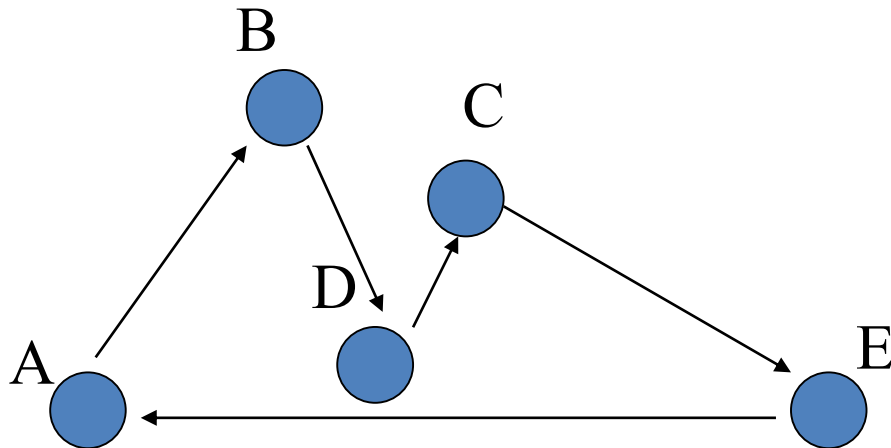
Find the shortest tour through the cities.

You must:

- Visit every city on the 'map'
- Return to where you started



The one below is length: 33



	A	B	C	D	E
A		5	7	4	15
B	5		3	4	10
C	7	3		2	7
D	4	4	2		9
E	15	10	7	9	

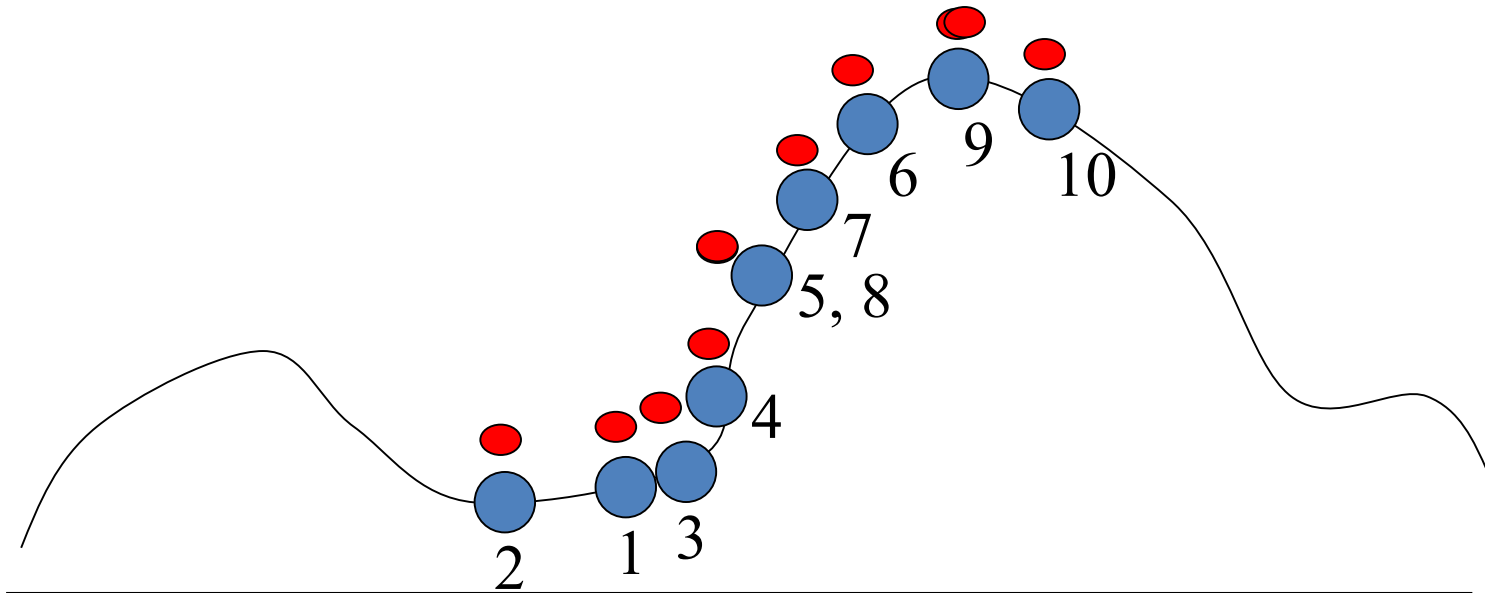
Hillclimbing

0. Initialise: Generate a random solution c ; evaluate its fitness, $f(c)$. Call c the *current solution*.
1. Mutate a copy of the current solution - call the mutant m . Evaluate fitness of m , $f(m)$.
2. If $f(m)$ is no worse than $f(c)$, then replace c with m , otherwise do nothing (effectively discarding m).
3. If a termination condition has been reached, stop. Otherwise, go to 1.

Note. No population (well, population of 1). This is a very simple version of an EA, although it has been around for much longer.

Why “Hillclimbing”?

Suppose that solutions are lined up along the x axis, and that *mutation* always gives you a nearby solutions. Fitness is on the y axis; this graph is a **landscape**



1. Initial solution; 2. rejected mutant; 3. new current solution,
4. New current solution; 5. new current solution; 6. new current soln
7. Rejected mutant; 8. rejected mutant; 9. new current solution,
10. Rejected mutant, ...

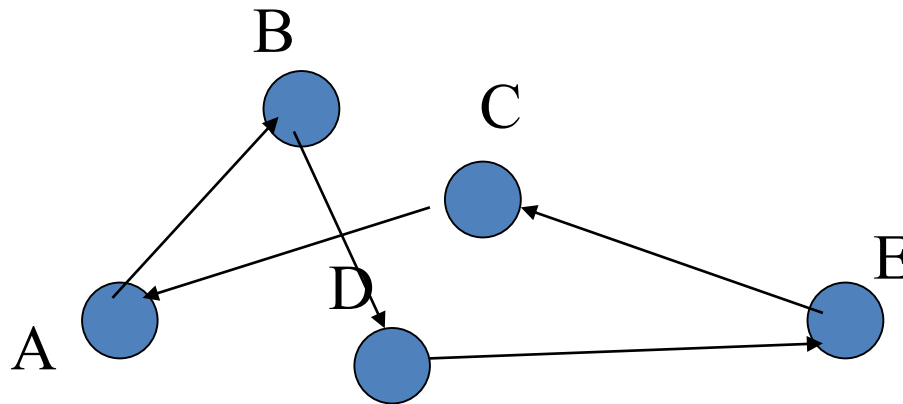
Example: HC on the TSP

We can encode a candidate solution to the TSP as a permutation

Here is our initial random solution ABDEC with fitness 32

	A	B	C	D	E
A		5	7	4	15
B	5		3	4	10
C	7	3		2	7
D	4	4	2		9
E	15	10	7	9	

Current solution

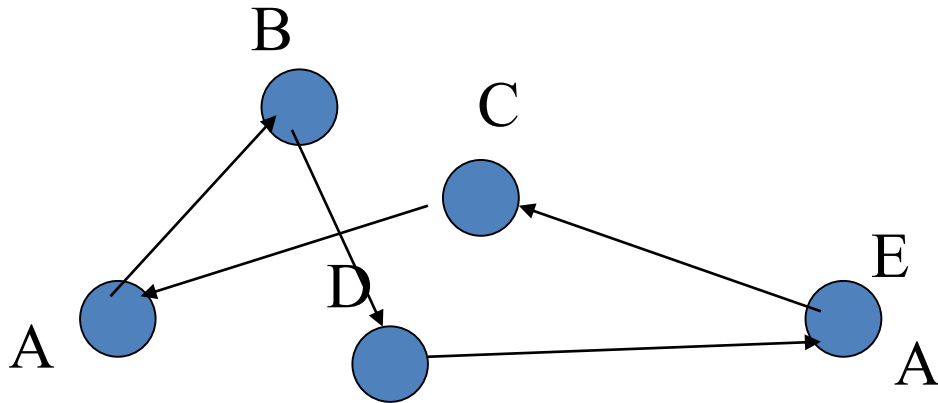


HC on the TSP

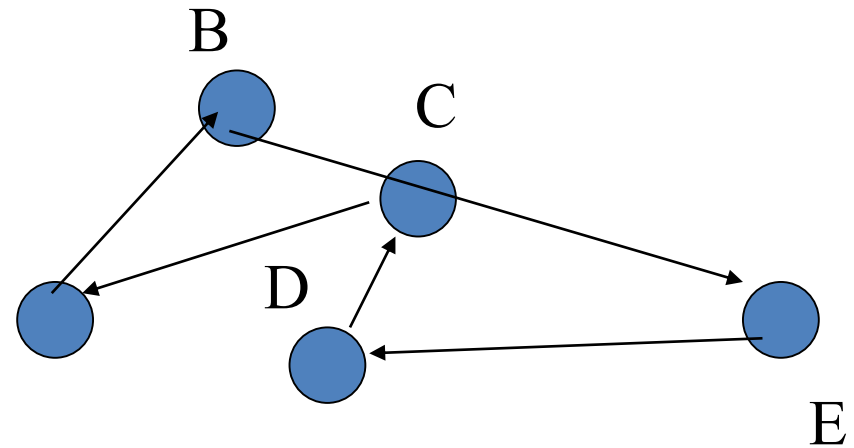
We randomly mutate (swap randomly chosen adjacent nodes) current (ABDEC, fitness 32) to: ABEDC, which has fitness 33
-- so current stays the same

	A	B	C	D	E
A		5	7	4	15
B	5		3	4	10
C	7	3		2	7
D	4	4	2		9
E	15	10	7	9	

Current solution



Mutant

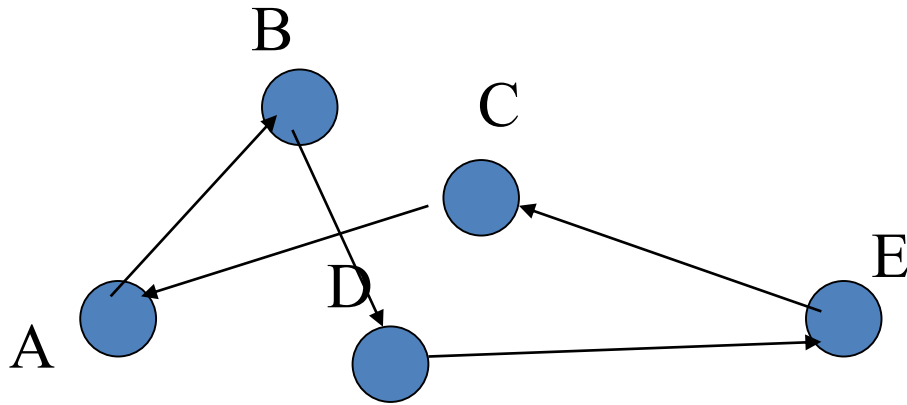


HC on the TSP

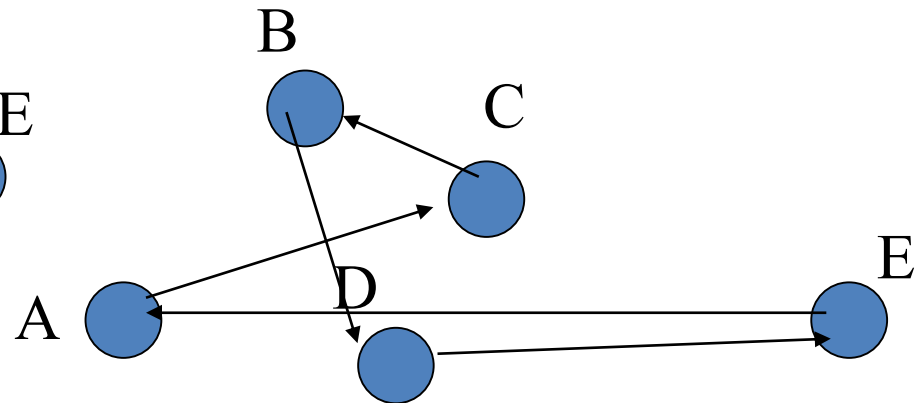
We randomly mutate (swap randomly chosen adjacent nodes) current (ABDEC, fitness 32) to **CBDEA**, which has fitness 38
-- so current stays the same

	A	B	C	D	E
A		5	7	4	15
B	5		3	4	10
C	7	3		2	7
D	4	4	2		9
E	15	10	7	9	

Current solution



Mutant



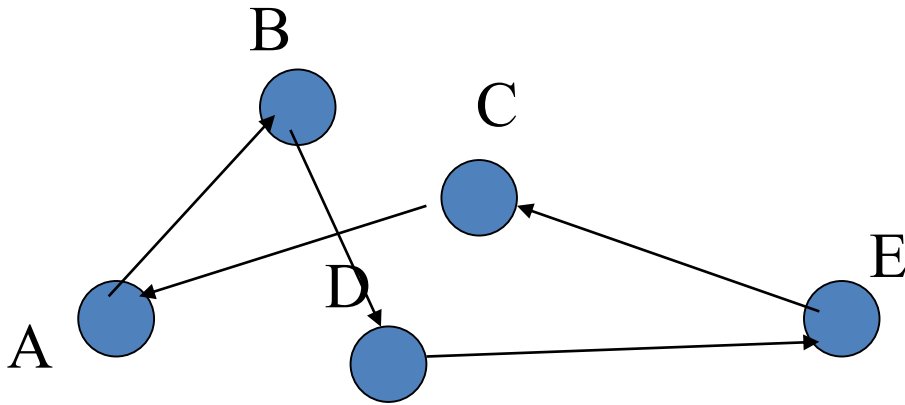
HC on the TSP

We randomly mutate (swap randomly chosen adjacent nodes) current (ABDEC, fitness 32) to **BA**DEC, which has fitness 28

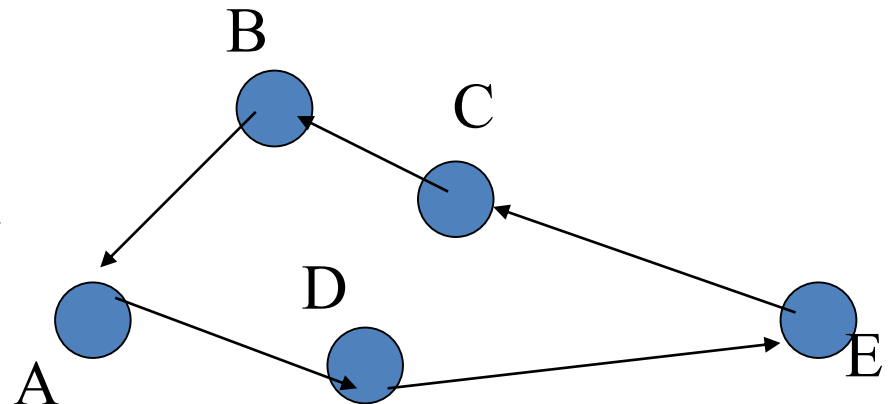
So this becomes the new current solution

	A	B	C	D	E
A		5	7	4	15
B	5		3	4	10
C	7	3		2	7
D	4	4	2		9
E	15	10	7	9	

Current solution



Mutant



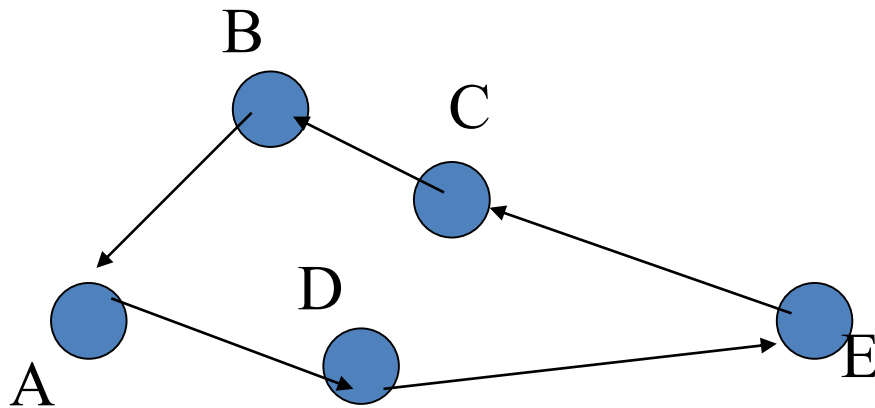
HC on the TSP

We randomly mutate (swap randomly chosen adjacent nodes) current (BADEC, fitness 28) to BADCE which also has fitness 28

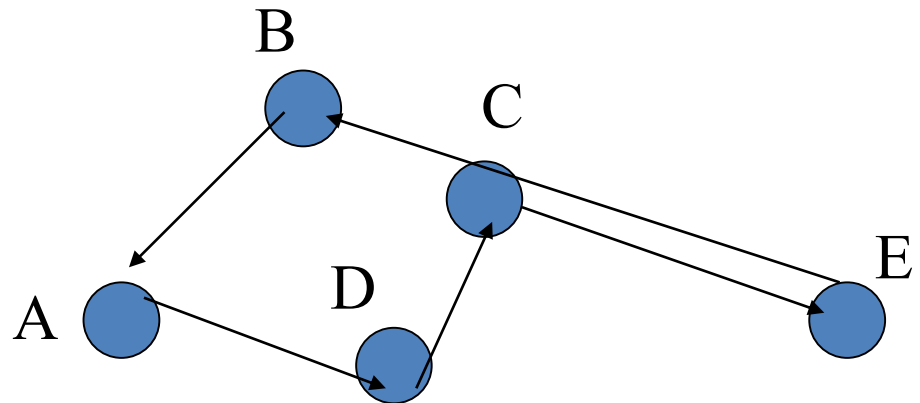
This becomes the new current solution

	A	B	C	D	E
A		5	7	4	15
B	5		3	4	10
C	7	3		2	7
D	4	4	2		9
E	15	10	7	9	

Current Solution

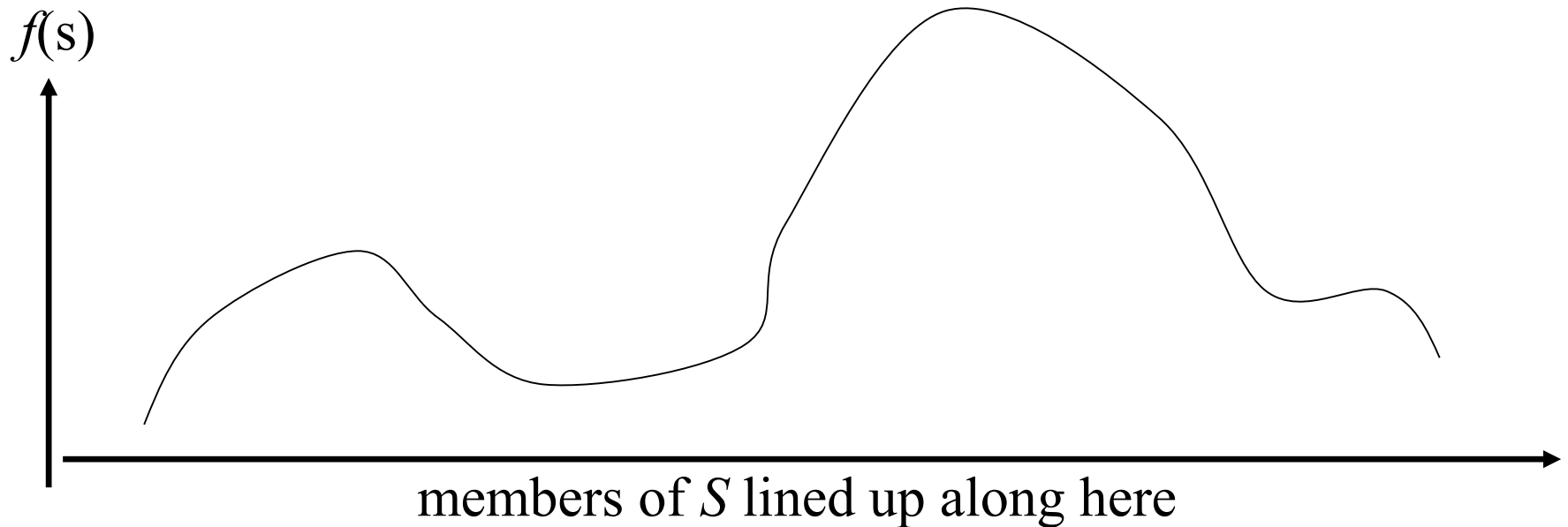


Mutant



Landscapes

Recall S , the search space, and $f(s)$, the fitness of a candidate in S



The graph we get by imposing $f(s)$ on S is called a *landscape*

What does the landscape look like if $f(s)$ is a random number generator?

What kinds of problems would have very smooth landscapes?

What is the importance of the mutation operator in all this?

Neighbourhoods

Let s be an individual in S , $f(s)$ is our fitness function, and M is our mutation operator, so that $M(s_1) \rightarrow s_2$, where s_2 is a mutant of s_1 .

Given M , we can usually work out the *neighbourhood* of an individual point s – the neighbourhood of s is the set of all possible mutants of s

E.g. **Encoding:** permutations of k objects (e.g. for k -city TSP)

Mutation: swap any adjacent pair of objects.

Neighbourhood: Each individual has k neighbours. E.g.

neighbours of EABDC are: {**A**EBDC, E**B**ADC, EA**D**BC, EAB**C**D, **C**ABDE}

Encoding: binary strings of length L (e.g. for L -item bin-packing)

Mutation: choose a bit randomly and flip it.

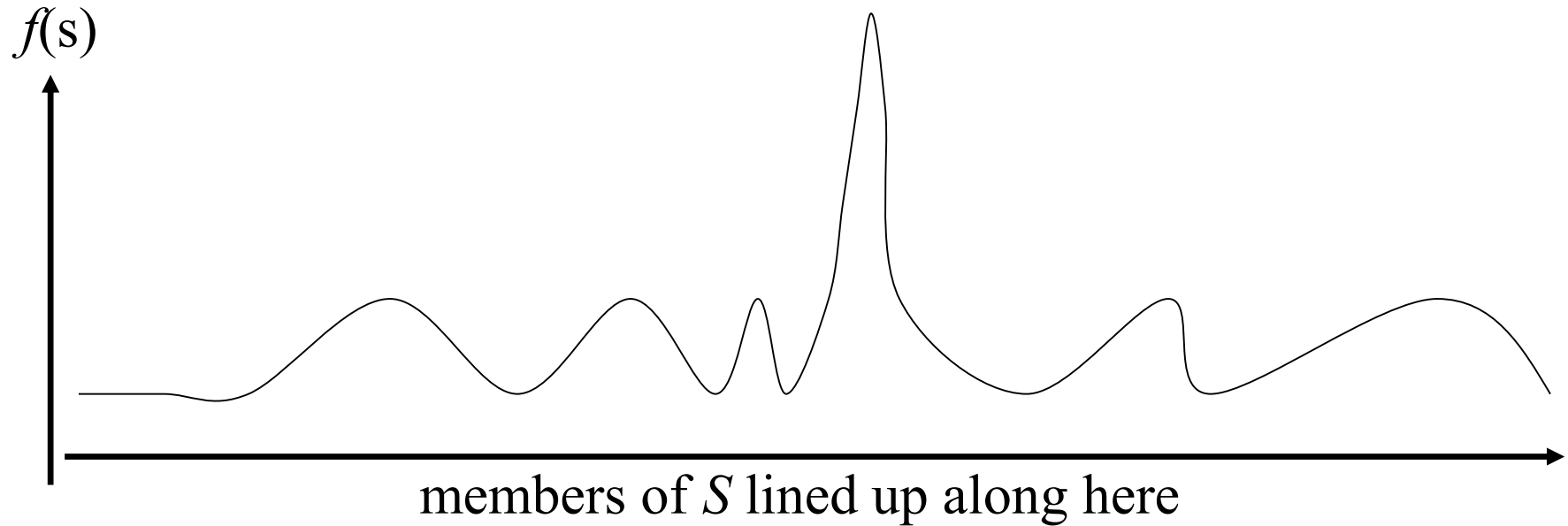
Neighbourhood: Each individual has L neighbours. E.g.

neighbours of 00110 are: {**1**0110, 0**1**110, 00**0**10, 001**0**0, 0011**1**}

Landscape Topology

- Mutation operators lead to slight changes in the solution, which tend to lead to slight changes in fitness.
I.e. the fitnesses in the neighbourhood of s are often similar to the fitness of s .
 - Landscapes tend to be *locally smooth*
- What about big mutations ??

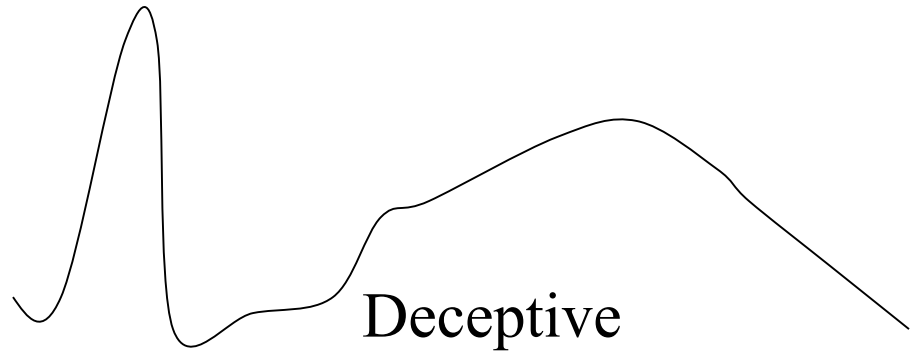
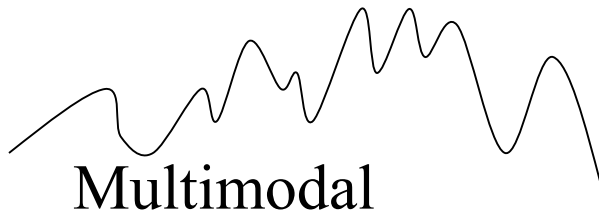
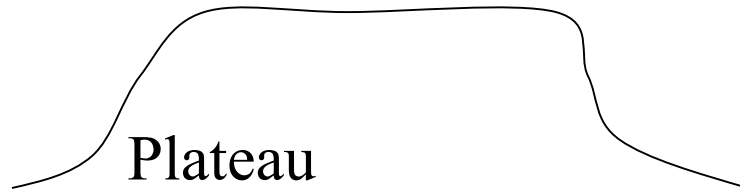
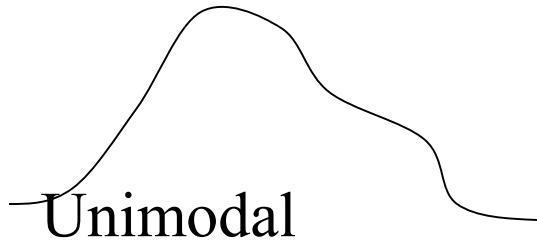
Typical Landscapes



Typically, with large (realistic) problems, the huge majority of the landscape has very poor fitness – there are tiny areas where the decent solutions concentrate.

So, big random changes are very likely to take us outside the good areas.

Typical Landscapes II



As we home in on the good areas, we can identify broad types of Landscape feature.

Most landscapes of interest are predominantly multimodal.

Despite being locally smooth, they are globally rugged

Beyond Hillclimbing

HC clearly has problems with typical landscapes:

There are two broad ways to improve HC, from the algorithm viewpoint:

1. Allow downhill moves – a family of methods called Local Search does this in various ways.
2. Have a population – so that different regions can be explored inherently in parallel – I.e. we keep ‘poor’ solutions in the population and give them a chance to ‘develop’.

Local Search

Initialise: Generate a random solution c ; evaluate its fitness, $f(s) = b$; call c the *current solution*, and call b the *best so far*.

Repeat until termination condition reached:

1. Search the *neighbourhood* of c , and choose one, m
 Evaluate fitness of m .
2. According to some policy, maybe replace c with m , and update c and b as appropriate.

E.g. Monte Carlo search: 1. same as hillclimbing (pick random m in the neighbourhood); 2. If $f(m)$ is better, accept m as new current sol; if $f(m)$ is worse, accept m with some probability (e.g. 0.1).

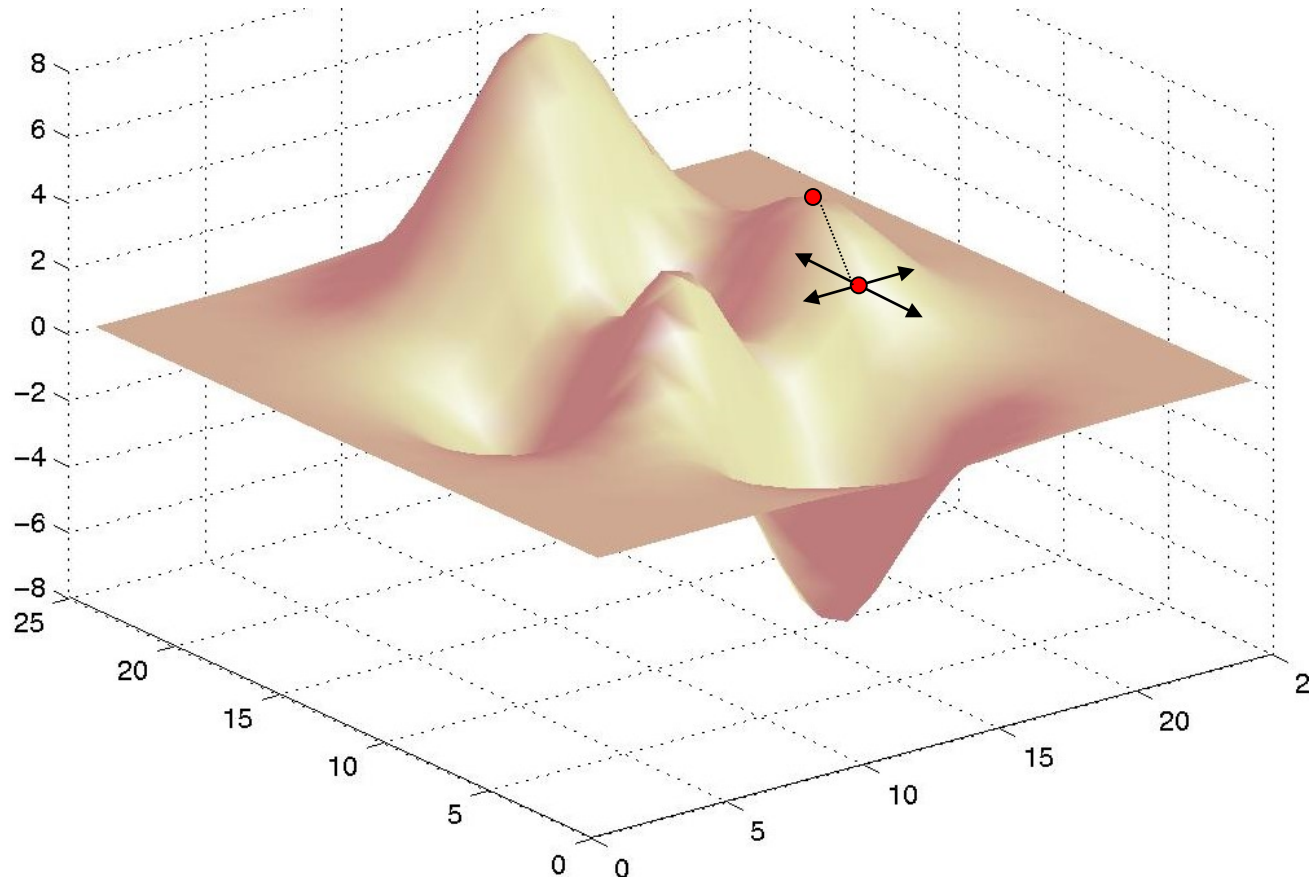
E.g. Tabu search: 1. evaluate *all* immediate neighbours of c ; 2. choose the best from (1) to be the next current solution (even if not improving), unless it is 'tabu' (recently visited), if so choose the next best, etc.

Population-Based Search

- Local search is fine, but tends to get stuck in *local optima*, less so than HC, but it still gets stuck.
- In PBS, we no longer have a single `current solution', we now have a *population* of them. This leads directly to the two main algorithmic differences between PBS and LS
 - Which of the set of current solutions do we mutate? We need a *selection* method
 - With more than one solution available, we needn't just mutate, we can [mate, recombine, crossover, etc ...] two or more current solutions.
- So this is an alternative route towards motivating our nature-inspired EAs – and also starts to explain why they turn out to be so good.

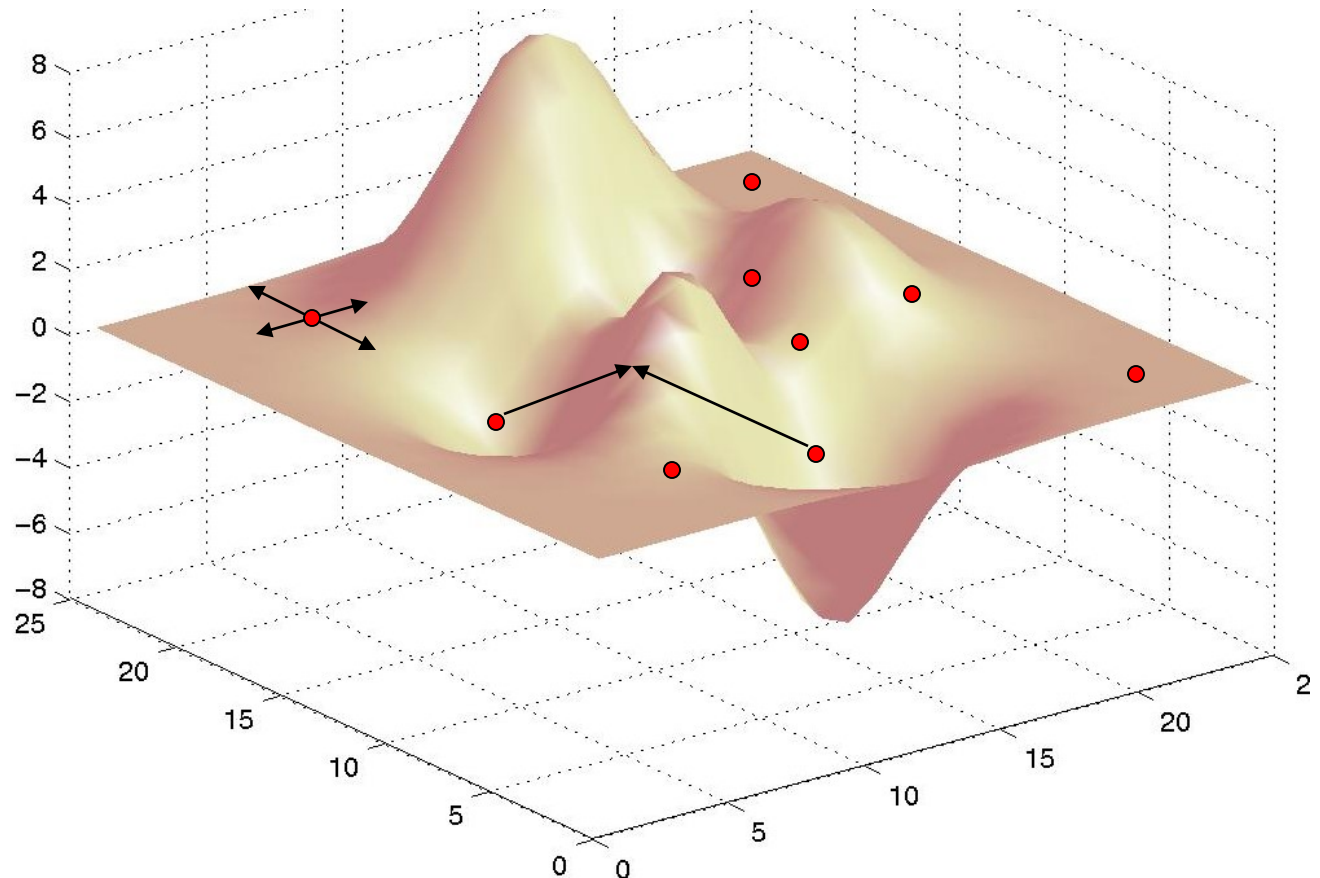
“Traditional” search/optimization algorithms and landscapes

- Gradient search and hill-climbing algorithms work by crawling up a fitness hill to the nearest local optimum – and getting stuck.



Evolutionary algorithms and landscapes

- EAs use populations of candidate solutions
- "Mutations" correspond to local moves in the fitness landscape
- "Crossovers" combine two candidate solutions



TSP, this time with an EA

A steady state EA with mutation-only, running for a few steps on the TSP example, with an unidentified selection method.

Running a Steady State EA -- TSP

	A	B	C	D	E
A		5	7	4	15
B	5		3	4	10
C	7	3		2	7
D	4	4	2		9
E	15	10	7	9	

Let's encode a solution as a permutation

Initial randomly generated pop of 5:

	ACEBD	DACBE	BACED	CDAEB	ABCED
Evaluation	32	33	32	31	28

Mutant of selected parent CDAEB → ADC^{red}EB

Evaluation of mutant: 26

Mutant enters population, replacing worst

Running a Steady State EA -- TSP

	A	B	C	D	E
A		5	7	4	15
B	5		3	4	10
C	7	3		2	7
D	4	4	2		9
E	15	10	7	9	

Generation 2

	ACEBD	ADCEB	BACED	CDAEB	ABCED
Evaluation	32	26	32	31	28

Mutant of selected parent BACED → BDCEA

Evaluation of mutant: 33

Mutant discarded— worse than current worst

Running a Steady State EA -- TSP

	A	B	C	D	E
A		5	7	4	15
B	5		3	4	10
C	7	3		2	7
D	4	4	2		9
E	15	10	7	9	

Generation 3:

	ACEBD	ADCEB	BACED	CDAEB	ABCED
Evaluation	32	26	32	31	28

Mutant of selected parent ABCED → AB**EC**D

Evaluation of mutant: 28

Mutant enters population, replacing worst

Running a Steady State EA -- TSP

	A	B	C	D	E
A		5	7	4	15
B	5		3	4	10
C	7	3		2	7
D	4	4	2		9
E	15	10	7	9	

Generation 4:

	ABECD	ADCEB	BACED	CDAEB	ABCED
Evaluation	28	26	32	31	28

Mutant of selected parent ADCEB → BDCEA

Evaluation of mutant: 33

Mutant is discarded

Running a Steady State EA -- TSP

	A	B	C	D	E
A		5	7	4	15
B	5		3	4	10
C	7	3		2	7
D	4	4	2		9
E	15	10	7	9	

Generation 5:

	ABECD	ADCEB	BACED	CDAEB	ABCED
Evaluation	28	26	32	31	28

Mutant of selected parent ABCED → AB**EC**D

Evaluation of mutant: 28

Mutant enters population, replacing worst

Running a Steady State EA -- TSP

	A	B	C	D	E
A		5	7	4	15
B	5		3	4	10
C	7	3		2	7
D	4	4	2		9
E	15	10	7	9	

Generation 6:

	ABECD	ADCEB	ABECD	CDAEB	ABCED
Evaluation	28	26	28	31	28

And so on.

Note: population starting to converge, genotypically and phenotypically

Conclusions

- There are a large number of variants of evolutionary algorithms, with different selection, mutation, crossover and replacement mechanisms
- Approaches such as hillclimbing and local search work up to a point
- However, they are prone to getting stuck in local minima
- Population-based search (such as that used in EAs) is better at avoiding this

- Readings:
 - An Introduction to Genetic Algorithms
by M. Mitchell
Chapter 1
 - Design of Modern Heuristics
by F. Rothlauf
Ch 2 (Landscapes),
Ch 5 (Local & Population-based search)
- Next Lecture: EAs in Details

